Chair of Software Engineering
TUM School of Computation, Information and Technology
Technical University of Munich

Advanced Software Testing and Analysis / Summer Semester 2026

Practical Project Tasks 01: Static Code Analysis

Prof. Stefan Wagner and Mohamed Ben Salha ✉

✉ mohamed.bensalha@tum.de

Publication: Monday, May 11, 2026, 08:00
Submission Deadline: Sunday, July 12, 2026, 23:59

Submission Instructions

- This exercise sheet has **3 pages**.
- The submission deadline for all exercise sheets is **Sunday, July 12, 2026, 23:59**.
- Submit your solutions **once** via Moodle before the submission deadline.
- Combine the solutions for **all** exercise sheets into a **single PDF file** and upload it to Moodle.
- Include your fork of the example project and any additional code/scripts as a **single ZIP file** and upload it to Moodle.
- This exercise is to be completed **individually**. Plagiarism will lead to not passing the project and may result in further disciplinary action.
- Grading is on a *pass*/*not passed* basis. You will receive feedback via Moodle after the submission deadline.
- If you pass, your exam grade will be improved by 0.3.
- Submissions are graded based on the following criteria
  - **Completeness**: All tasks are addressed.
  - **Correctness**: $\geq 80\%$ of tasks are solved correctly.

Support

Make sure to follow each step carefully and check the deliverable checklist before submitting your work. If you encounter any issues, use the Moodle discussion forum to ask questions.

Task 1 Set up the Example Project

Your first task is to set up and run a local copy of the example project.

a) Visit the GitLab repository of the example project[1] and click on the "Fork" button to create your own copy of the repository under your GitLab account. Then, use `git clone` to get a local instance of your fork to run the project from your local instance.

b) Build the project using Maven[2] and the provided `pom.xml` file. Briefly describe the steps you took to get the example project running on your machine. Include any issues you encountered and how you resolved them. Take a screenshot of your terminal showing the console output upon running the project.
Hint 1: Building the project may involve running a command such as `mvn clean install`.
Hint 2: The output should indicate a build, displaying a message like `[INFO] BUILD SUCCESS` or `[INFO] BUILD FAILURE` or `[ERROR] Tests run: 391, Failures: 1, Errors: 0, Skipped: 0`.

c) Finally, edit the `README.md` file and add your name, student e-mail, and TUM ID to the top of the file. Commit the changes locally and push them to your fork on GitLab.

Deliverable Checklist

- ☐ **PDF**: Screenshot(s) of the **console output** upon a build of the example project using Maven.
- ☐ **PDF**: A text description of the steps you took to **get the example project running**. This should include any setup steps, issues encountered, and how they were resolved.
- ☐ **ZIP**: Provide the source code of your **repository fork** as a ZIP file. Ensure the ZIP file includes all project files and your changes to the `README.md` file. You only need to add the repository ZIP once per submission.

Task 2 Set up a Static Code Analysis Tool

Next, you should set up a static code analysis tool to analyze the code base of the project. For this purpose, we recommend that you run the community edition of SonarQube[3] with Docker. However, you may use any other static code analysis tool that is compatible with the example project and at least provides one code quality metric (e.g. cyclomatic complexity) and code smell, issue, or vulnerability detection.

a) Install and set up the static code analysis tool of your choice and analyze the code of the example project.
Hint: To analyze a code base with SonarQube, you need to set up a local instance of SonarQube and install a version of the SonarScanner. For the example project, you can use the Maven SonarScanner or the SonarScanner CLI.

b) Briefly describe the main elements of the analysis report overview and include a screenshot.

c) Briefly describe how you ran the static code analysis tool and how the resulting report can be replicated. This should include the installation and setup process, configuration of the analysis tool, and execution of the analysis. Also, include any issues you encountered and how you resolved them.

d) Add any additional code, scripts, or configuration files you created during the process of running the static code analysis to your submission ZIP. Update the `README.md` file to point out any additional files you added to the repository. Describe their purpose and how they are used.

Deliverable Checklist

- ☐ **PDF**: Screenshot(s) of the **analysis report overview** upon the completed analysis of the project.
- ☐ **PDF**: A text description of the different elements of the **analysis report overview**.
- ☐ **PDF**: A text description of the steps you took to **run the static code analysis** and how the resulting report can be replicated.
- ☐ **ZIP**: Provide any additional code, scripts, or configuration files you created during the process of running the static code analysis. Ensure the ZIP file includes all relevant files and your updated `README.md` file. Make sure to point out in your `README.md` what additional files you added. You only need to add the repository ZIP once per submission.

[1] https://gitlab.lrz.de/tobias-eisenreich/sirius-kernel
[2] https://maven.apache.org/
[3] https://docs.sonarsource.com/sonarqube/latest/try-out-sonarqube/

Task 3 Analyze the Code Analysis Results

Your final task is to analyze the analysis results.

a) Choose a **code metric or measure** reported by the static analysis tool, e.g. cyclomatic complexity. Explain how the metric is calculated. Attach a screenshot of the metric values found for a class of your choosing in the analysis report. Ensure the class name and metric values are clearly visible in the screenshot. What do the calculated values mean for the software quality of the project? Discuss what the analysis results indicate about the software quality of the project.

b) Choose three occurrences of different **code smells, issues, or vulnerabilities** reported by the static analysis tool. For each issue, describe what the issue means and attach a screenshot of an occurrence found in the analysis report. Ensure that the specific issue name and its location in the code are clearly visible in the screenshot. What does the occurrence of the issue mean for the software quality of the project? Discuss what the analysis results indicate about the software quality of the project.

Deliverables

1. **PDF**: Screenshot(s) of the analysis results for your chosen **code metric or measure** for your chosen class. A text description of how the metric is calculated and what the analysis results indicate for the software quality of the example project.
2. **PDF**: Screenshot(s) of the analysis results for your chosen **three code smells, issues, or vulnerabilities**. A text description of what each issue means and what the analysis results indicate for the software quality of the example project.